

Introduction to Machine Learning

Review Notes

This is the first true paradigm shift in the series. In the first five chapters, you wrote explicit rules with `if/elif`, Z-scores, and `apply`. From this chapter onward, the model **learns rules from data**. You provide inputs and correct answers instead of hand-writing the decision logic. The core is one fixed six-step pipeline and three questions that unfold around it.

- Python 3 - scikit-learn
- tutor style
- visual explanations
- FAQ included
- six-step workflow

00 The Six-Step Pipeline and Three Follow-Up Questions

The whole chapter is one stable six-step workflow, followed by three questions about what the model learned.

Step	What to do
1. Read the data	<code>pd.read_csv</code>
2. Separate what the model sees from what it must guess	<code>X = df[feature_cols] , y = df["label"]</code>
3. Keep some data hidden from the model	<code>train_test_split(X, y, test_size, random_state)</code>
4. Perform the real learning step	<code>model.fit(X_train, y_train)</code>
5. Ask the model to guess and compare with the answer key	<code>model.predict(X_test)</code> + <code>accuracy_score</code>
6. Use the trained model on unseen data	wrap one row in a DataFrame, then predict

What did the model learn?
Which features drive decisions?

Where does it fail?
Which samples are misclassified?

Is the accuracy trustworthy?
Did it memorize instead?

01 - START
X and y
inputs vs answers

02
train_test_split
hold out a test set

03
fit / predict
learn / use

04
Decision tree
a chain of yes/no

05
New sample
a two-dimensional row

06
Switch models
no universal best

07
Error inspection
reuse Pandas

08
Importance
which inputs matter

09
Overfitting
memorization vs learning

10
max_depth
control complexity

11
Model sweep
compare hyperparameters

The paradigm shift: earlier, you wrote a rule such as `if score > 5: "urgent"` and chose the threshold yourself. Here, an algorithm derives thresholds from data. A decision tree is still a chain of conditional judgments, but the numbers now come from statistical learning rather than your hand-written rules.

Introduction to Machine Learning - Workflow 2.0

1 / 8

01 Supervised Learning: Features (X) and Label (y)

• Core concept

Supervised learning separates evidence from the answer

X (features)

heart_rate, oxygen_level, temperature
inputs the model is allowed to see

y (label)

risk_category
the answer to predict

Never mix X and y. Leaking the answer into the input is cheating.

Define X and y

x / y

```
X = dataframe[FEATURE_COLUMNS] # heart_rate, oxygen_level, temperature
y = dataframe["risk_category"] # the answer the model must learn to predict
```

The model must never see X and y mixed together. The label is the answer key. If it leaks into the input, the model is not learning; it is cheating by reading the answer. This echoes the earlier Pandas function `compute_deviation_score`: the inputs may be heart rate, oxygen, and temperature, but the output risk level must remain separate.

• Common mistake

Incorrect

Put `risk_category` inside `X` while also using it as `y`

Why it fails

The model reads the answer directly, producing nearly 100% but meaningless accuracy.

Correct

```
X = dataframe[FEATURE_COLUMNS] with no risk_category
```

02 train_test_split: Why Training Data Cannot Be the Exam

• Core concept

Hide 20% of the data from the model

split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# test_size=0.2    -> reserve 20%; do not show it during training
# random_state=42 -> repeat the same random train/test split
```

If training and testing use the same records, the model can memorize the answer key and score perfectly. A test set is a batch the model has never seen, so it is the only honest way to assess generalization. Splitting the data is a fixed step before training any model.

Q. Why set `random_state=42`? Is it the same as NumPy's random seed?

Yes, the idea is the same. `train_test_split` randomly decides which rows enter the training and test sets. Fixing `random_state` makes that split reproducible across machines and runs, which makes results comparable. The number 42 is conventional; any fixed integer works.

03 fit / predict / accuracy_score: The Three-Step Routine

• Core concept

Learn -> guess -> evaluate

fit / predict

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)           # learn only from the training set

predictions = model.predict(X_test)   # guess labels for unseen rows
accuracy_score(y_test, predictions)  # compare truth with guesses
```

The argument order is truth first, prediction second: `accuracy_score(y_true, y_pred)`. The result is a proportion from 0 to 1; for example, 0.833 means 10 of 12 test samples were classified correctly. Almost every classifier follows this same three-step routine.

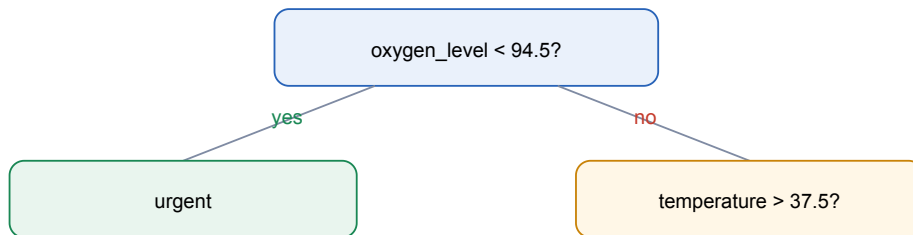
• Common mistake

Incorrect	Why it fails	Correct
<code>predictions = model.predict(X_train)</code> for evaluation	High accuracy may only mean the model memorized the training records.	Evaluate on <code>X_test</code> .

04 Decision Trees: A Chain of Yes-or-No Questions

• Core concept

A tree learns both the questions and their thresholds



You might choose 95 and 37.5; training found 94.5 and 37.5 from the data.

Decision tree = sequential threshold decisions

tree

```
model = DecisionTreeClassifier(random_state=42)
```

A decision tree resembles hand-written `if/elif` logic: ask whether oxygen is below 94.5, then perhaps whether temperature exceeds 37.5. The difference is where the thresholds come from. You chose 95 and 37.5 by experience; the tree searches its training data for splits and decides what to ask and in what order. Trees are useful when relationships are not simply linear and the result should remain interpretable.

05 Predicting One New Patient with a Trained Model

• Core concept

A single sample must still be a two-dimensional table

predict

```
row = pd.DataFrame(
    [[heart_rate, oxygen_level, temperature]],
    columns=FEATURE_COLUMNS,
)
model.predict(row)[0]
```

scikit-learn's `predict` always expects a two-dimensional table because even one person is a dataset with one row. Do not pass three bare numbers. `predict` also returns an array; when there is exactly one result, use `[0]` to extract that value.

• Common mistake

Incorrect	Why it fails	Correct
<pre>model.predict([130, 89, 39.0])</pre>	A one-dimensional list has the wrong shape; sklearn expects rows by columns.	<pre>pd.DataFrame([[130, 89, 39.0]], columns=FEATURE_COLUMNS)</pre>

06 Switching Models: Accuracy Is Not a Universal Truth

• Core concept

Same data, different model, different fit

[compare](#)

```
model = LogisticRegression(max_iter=1000, random_state=42)
# With the same X, y, and split, LogisticRegression may score 0.75
# while the decision tree scores 0.833.
```

Neither result means one implementation is wrong. Different models have different built-in shapes: a decision tree learns a sequence of threshold rules, while logistic regression looks for a linear boundary that separates the classes. `max_iter=1000` gives the optimizer enough iterations to converge. There is no universally best model; try several appropriate models and prefer the one that generalizes better on the same held-out data.

07 Inspecting Errors: Where the Model Fails Matters

• Core concept

Reuse familiar Pandas operations

[inspect](#)

```
results = X_test.copy()
results["actual"] = y_test.values
results["predicted"] = model.predict(X_test)
results["correct"] = results["actual"] == results["predicted"]

mask = results["correct"] == False
misclassified = results[mask]
```

No new syntax appears here. Copy a table, add columns, and filter with a Boolean mask - the same Pandas skills as before, now applied to model predictions. `y_test.values` converts the label Series into a plain array so Pandas does not realign it by the original shuffled index. Studying errors often teaches more than one accuracy number: confusing watch with urgent is different from confusing normal with urgent.

• Code template

Template 2 - table of prediction errors

[misclassified](#)

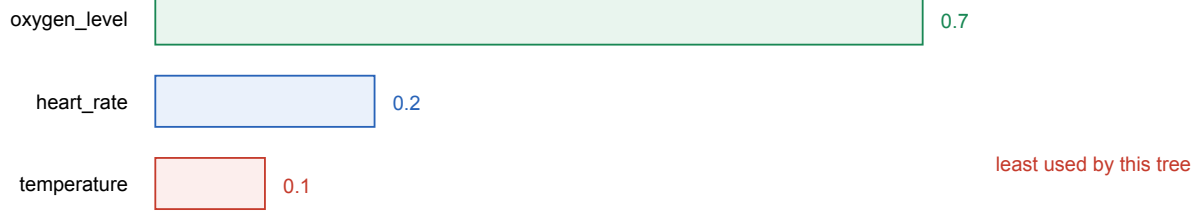
```
def build_results_table(model, X_test, y_test):
    results = X_test.copy()
    results["actual"] = y_test.values
    results["predicted"] = model.predict(X_test)
    results["correct"] = results["actual"] == results["predicted"]
    return results

def misclassified(results):
    return results[results["correct"] == False]
```

08 feature_importances_: Which Inputs Drive the Model?

• Core concept

Importance in this particular trained decision tree



The training examples rarely crossed the temperature threshold, so the tree used it less.

An attribute, not a method

importance

```
importances = pd.Series(model.feature_importances_, index=feature_names)
importances.sort_values(ascending=False)
```

`model.feature_importances_` is an attribute learned during tree training, not a function, so it has no parentheses. It returns a NumPy array whose length equals the number of features and whose values sum to 1. The essential warning is that importance reflects how much a feature helped this tree classify this training set. It is not the same as the feature's theoretical or medical importance.

• Common mistake

Incorrect

Why it fails

Correct

`model.feature_importances_()`

Parentheses treat an attribute as if it were a method.

`model.feature_importances_`

Q. If `feature_importances_` ranks a feature first, does that mean it is medically most important?

No. Temperature may carry the largest weight in a hand-written clinical rule but appear least important in this tree because few training examples cross the fever threshold. `feature_importances_` reports actual contribution to this classification task on this specific dataset. A new sample or distribution may produce a completely different ranking.

09 Overfitting: Training Accuracy Is Not Real-World Ability

• Core concept

Training 1.0 vs test 0.833 - memorization is not understanding

overfit

```
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)
train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)
```

An unrestricted tree may score 1.0 on all 48 training rows but only 0.833 on the test set. It did not suddenly become worse; it memorized the training records, including noise and coincidences, instead of learning rules that transfer. This is **overfitting**. The defining symptom is a large train-test gap, not merely high training accuracy.

• Common mistake

Incorrect

Why it fails

Correct

Report only training accuracy = 1.0

It shows memory of training data, not the ability to generalize.

Report both train and test accuracy and compare the gap.

10 max_depth: Control Overfitting with a Question Limit

• Core concept

Depth	Observed pattern	Conclusion
Too shallow, such as 1	Training and test accuracy are both low.	Underfitting
Moderate	Training and test accuracy are close.	A better balance point
Too deep or unlimited	Training approaches 1.0 while test accuracy stops improving or falls.	Overfitting; the gap indicates its severity

Limit how many layers of yes/no questions the tree can ask

max_depth

```
model = DecisionTreeClassifier(
    random_state=42, max_depth=max_depth
)
```

max_depth limits how many successive layers of questions the tree may use before it must produce an answer. Without a limit, a tree may keep splitting until individual training records are isolated. Limiting depth encourages a rougher rule that can generalize better to unseen data.

Q. How do train and test accuracy distinguish underfitting from overfitting?

Low train and low test accuracy suggest underfitting: the model is too simple. High train and high test accuracy with a small gap is the desired balance. High train but clearly lower test accuracy suggests overfitting. Do not read only the absolute scores; read their difference, because a larger gap means more severe overfitting.

11 Train Several Models in a Loop for Comparison

• Core concept

The one place in this chapter where an ordinary for loop is appropriate

sweep

```
rows = []
for depth in depths:
    model = train_shallow_tree(X_train, y_train, max_depth=depth)
    train_acc, test_acc = train_test_accuracy(
        model, X_train, y_train, X_test, y_test
    )
    rows.append({"depth": depth, "train_accuracy": train_acc,
                "test_accuracy": test_acc})
return pd.DataFrame(rows)
```

A regular loop is correct because each iteration trains a new model. This is not repeated arithmetic over one array; it is a loop over separate experiments with different states. Collect the rows into a DataFrame, then use Matplotlib to plot depth against training and test accuracy.

• Common mistake

Incorrect	Why it is not ideal	Correct
<pre>for row in X_test.itertuples(): predict_one_row(row)</pre>	predict is already vectorized and can process the entire table at once.	<pre>predictions = model.predict(X_test)</pre>

Q. Why is looping across tree depths correct, but looping across rows for prediction wrong?

Ask what the loop is doing. If every iteration repeats the same simple calculation over rows, such as encoding or prediction, the library already supports vectorization. If every iteration creates a completely different object, such as a new tree with a different depth, then it represents separate experiments and a normal loop is clear and appropriate.

Common Error Summary

Incorrect	Why it fails	Correct
Put the label column in X	The model reads the answer and produces meaningless accuracy.	<code>X = dataframe[FEATURE_COLUMNS]</code>
Evaluate on the training set	A high score may only reflect memorization.	Evaluate on <code>X_test</code> .
<code>model.predict([130, 89, 39.0])</code>	The input is one-dimensional.	Wrap it as a one-row DataFrame.
Conclude from training accuracy alone	It cannot establish generalization.	Compare the train-test gap.
<code>model.feature_importances_()</code>	This is an attribute, not a method.	Remove the parentheses.
Predict one row at a time in a loop	<code>predict</code> is already vectorized.	Predict the full batch once.

FAQ

Q. What is the essential difference between a decision tree and my hand-written if/elif rules?

The structure is similar: both are chains of conditional questions ending in a classification. The difference is where the thresholds come from. You choose 95 and 37.5 from experience; a decision-tree algorithm searches the training data for thresholds that best separate the classes.

Q. Does lower accuracy always mean a model is useless?

No. The same dataset may yield 0.833 for a tree and 0.75 for logistic regression without an absolute pass line. Interpret accuracy relative to other suitable models, random guessing, task cost, and domain requirements. Training accuracy alone is especially misleading.

Q. Why is looping over several tree depths correct while looping over every row is wrong?

The distinction is the work performed by each iteration. Repeating the same simple operation on rows should be vectorized. Building a completely new model at every depth creates separate experiments, so a normal loop is the right abstraction.

Exercises

• Practice problems

- E** **Separate X and y:** the columns are `age`, `blood_pressure`, and `diagnosis`; the target is diagnosis. Expected: `X=data[["age","blood_pressure"]]`, `y=data["diagnosis"]`. Practice keeping the label out of X.
- M** **Complete the standard six-step pipeline:** given `labeled_patients.csv`, write every step from loading data to predicting a new patient. Practice the correct order and method name for each stage.
- E** **Find misclassified patients:** given a `results` table with `actual` and `predicted`, write one line that returns all incorrect rows. Expected: `results[results["actual"] != results["predicted"]]`.
- M** **Diagnose overfitting:** `train_accuracy=0.98`, `test_accuracy=0.65`. Decide whether this is underfitting, balanced, or overfitting and suggest an adjustment. Expected: overfitting; reduce `max_depth`.
- M** **Rank feature importance:** `feature_importances_[0.1,0.7,0.2]` for `[heart_rate,oxygen_level,temperature]`. Write code that returns a descending ranking. Practice `pd.Series(...).sort_values(ascending=False)`.

Review Order and Chapter Checklist

• Must master

Separate features `X` from label `y`; the label never enters model input

`train_test_split(X, y, test_size, random_state)` creates an honest test set

`fit` means learn; `predict` means use; never evaluate both on the same data

`accuracy_score(y_true, y_pred)` argument order

A single new sample must be a one-row `DataFrame`

Detect overfitting with the train-test accuracy gap

• Recommended practice

Decision tree vs logistic regression: no universally best model

`feature_importances_` reports contribution on this dataset

Use familiar Pandas operations to inspect prediction errors

`max_depth` controls the balance between underfitting and overfitting

Know when a loop represents experiments rather than row-wise work

• Next steps

More classifiers: random forest / SVM / KNN

Cross-validation

Confusion matrix, precision, and recall

Feature engineering

Regression for continuous targets

Reusable project templates

• Chapter checklist

- ☐ Features `X` and label `y` are strictly separated.
- ☐ A test set is reserved before training.
- ☐ One new sample is still a two-dimensional `DataFrame` row.
- ☐ `feature_importances_` is an attribute describing this trained tree.
- ☐ Overfitting is diagnosed by comparing training and test accuracy.
- ☐ The label never appears in model input.
- ☐ `fit`, `predict`, and `accuracy_score` follow the correct order.
- ☐ Different models may behave differently on the same data.
- ☐ Prediction errors can be inspected with familiar Pandas masks and columns.
- ☐ `max_depth` balances underfitting against overfitting; model sweeps are vectorized experiments, not row loops.

After every exercise: you have completed the most basic and most important machine-learning workflow - from a dataset to a trained model, with a way to diagnose whether it learned anything real. This is the key step from data processing into AI/ML.